14 (A) - Learning MNL Model

IEOR 145/245

Rajan Udwani

Reminders

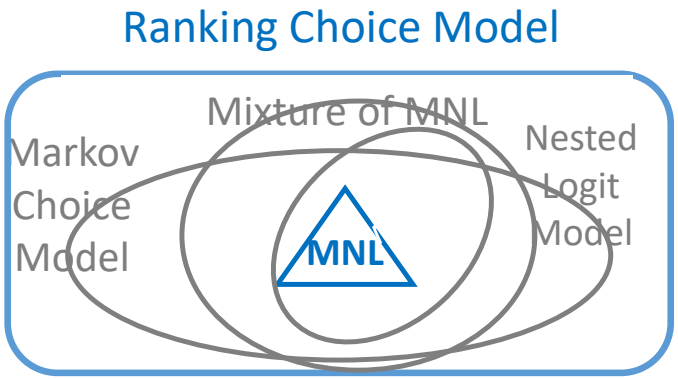
- Please **meet with your teams to figure out course project topic**
  - Due next week
  - Please read the project README file carefully if you haven't yet
  - I will end early today to give you time to chat with me
- Group simulation assignment will be released today
- Practice exam and other details about exam will be released by Thursday

Multinomial Logit Choice (MNL)

v_j: weight of product $j \in [n]$

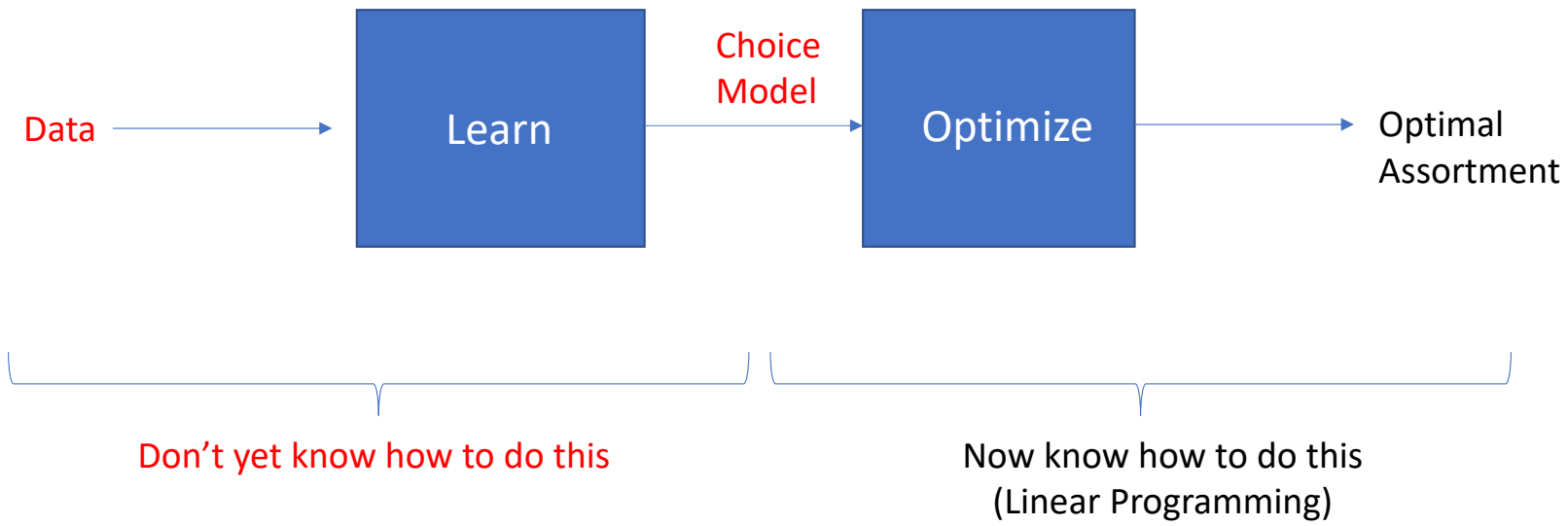
v_0: weight of outside option

$$\phi(i, S) = \frac{v_i}{v_0 + \sum_{j \in S} v_j}$$



- Optimization and learning are tractable
- Often the go-to model, but has clear limitations

Recall: Data to Decision Pipeline



Recall: Maximum Likelihood Estimate for A Coin

- Suppose probability of heads is p
- Then, what is probability of getting H H H H H T H T H H ?

$$\text{Likelihood(p)} = p^8(1-p)^2$$

- What value of p maximizes likelihood?

$$p^* = \text{argmax}_{p\in[0,1]}\, p^8(1-p)^2$$

Recall: What Would Data Look Like?

- Typical sample consists of the assortment shown and choice made

S_t, i_t: customer t saw assortment S_t and chose $i_t \in S_t \cup \{0\}$

- Available in brick and mortar stores as well as online platforms
  - Online platforms also have other data such as “clicks” or “hover”

- Dataset is a collection of **independent** samples $\{S_t, i_t\}_{t \in T}$
  - Called *sales data*

Given a Single Sample S_t, i_t

Example: $\{1,2,3\}, 1$

- Are all MNL models viable for this sample?
- Models with $v_1 = 0$ are not viable
- Most likely model is,

$$\text{argmax}_{(v_1,v_2,v_3)}\ \phi(1,\{1,2,3\}) = \text{argmax}_{(v_1,v_2,v_3)}\ \frac{v_1}{1+v_1+v_2+v_3}$$

Only relative weights matter: ($v_0 = 1$ as part of normalization)

Learning MNL Model

- Given sales data $\{S_t, i_t\}_{t \in T}$

- **Maximum Likelihood Estimate** $= \mathbf{argmax}_{(v_i)} \prod_t \boldsymbol{\phi}(\boldsymbol{i_t}, \boldsymbol{S_t})$

Probability of sample t

Combined probability of all samples

Learning MNL Model

- Given sales data $\{S_t, i_t\}_{t \in T}$
- Maximum Likelihood Estimate $= \operatorname{argmax}_{(v_i)} \prod_t \phi(i_t, S_t)$
- Log-likelihood estimate $= \operatorname{argmax}_{(v_i)} \sum_t \log \phi(i_t, S_t)$
- Maximizing likelihood same as maximizing log-likelihood
  - log is monotonically increasing

Why Take Logs?

- A product of many probabilities gets tiny very fast
  - 100 samples, each with probability 0.5: $0.5^{100} \approx 7.9 \times 10^{-31}$
- The log turns the product into a sum
  - $100 \times \log 0.5 \approx -69.3$: numerically better to work with
- Log is increasing so the maximizer does not change
  - Coin example: $p^8(1-p)^2$ and $8 \log p + 2 \log(1-p)$ both peak at $p = 0.8$

Learning MNL Model

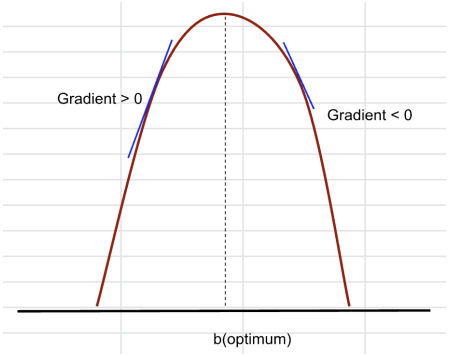
- Given sales data $\{S_t, i_t\}_{t \in T}$
- Log-likelihood estimate

$$= \text{argmax}_{(v_i)} \sum_t \log \frac{v_{i_t}}{1 + \sum_{i \in S_t} v_i}$$

($v_0 = 1$ as part of normalization)

$$= \text{argmax}_{(v_i)} \sum_t \log v_{i_t} - \sum_t \log \left(1 + \sum_{i \in S_t} v_i\right)$$

- Concave function, gradient ascent finds a solution efficiently



Log-Likelihood: Summary

1. Each sample t adds two terms to the log-likelihood
   - $+\log v_i$ for the chosen item $i=i_t$
   - $-\log(1+\sum v_i)$, summed over the items shown in S_t

2. $v_0=1$ because only relative weights matter
   - $(v_0,v_1,v_2)=(1,2,1)$ and $(2,4,2)$ give the same choice probabilities

3. Concave, which implies one single hill: keep walking uphill (gradient ascent) in sufficiently small steps until you reach the top

Code available in “Slides-> MNL_code.zip”

Feel free to use for the simulation exercise

```
[26]: (array([93, 89, 79, 63, 19, 21, 81, 48, 86, 38]), 22.986771343989016)
```

Assortment output by pipeline (data to decision)

```
[27]: def real_expected_revenue(ground_truth, prices, assortment):

          assortment_indices = (assortment - np.ones(10)).astype(int)
          assortment_true_parameters = ground_truth[assortment_indices][:,0]
          probs = assortment_true_parameters / (sum(assortment_true_parameters) + 1)
          revenue = sum(probs * prices[assortment_indices])
          return revenue
```

```
[28]: #real revenue for calculated optimal assortment
      real_expected_revenue(ground_truth, item_prices, estimated_optimal_assortment)

[28]: 22.995186152121857
```

Revenue (data to decision)

```
[29]: #the prices of the items we have selected
      item_prices[(estimated_optimal_assortment - np.ones(10)).astype(int)]

[29]: array([25, 25, 25, 23, 24, 23, 24, 23, 25, 23])

[30]: #the estimated values of the items we have selected
      result[(estimated_optimal_assortment - np.ones(10)).astype(int)]

[30]: array([9.88972190e-14, 1.13969379e-13, 1.51370267e-13, 8.07531387e+00,
      ,       9.81224460e-01, 2.16292689e+01, 4.06349503e+00, 5.09927808e+01,
      ,       4.42831672e+00, 5.96627896e+02])

[71]: #real expected revenue of a random assortment
      real_expected_revenue(ground_truth, item_prices, np.random.randint(100, size=10))

[71]: 15.272763144499502
```

Revenue of random assortment

```
[72]: #real expected revenue of the 10 items with highest price
      top10_expensive_assortment = [8, 14, 26, 43, 79, 82, 86, 88, 89, 93]
      print(item_prices[(top10_expensive_assortment-np.ones(10)).astype(int)])
      real_expected_revenue(ground_truth, item_prices, top10_expensive_assortment)

      [25 25 25 25 25 25 25 25 25 25]
[72]: 7.308712984898049
```

Revenue of highest price products

```
[64]: #equivalent to true optimal
      find_optimal_assortment(np.ravel(ground_truth), item_prices)

[64]: (array([79, 18, 66, 19, 86, 63, 81, 21, 48, 38]), 22.995187592074824)
```

True optimal (if we knew choice model exactly)

Important Question (To be answered)

What if data does not have item e, i.e., e is not shown in any of $\{S_t\}_{t \in T}$?

Log-likelihood estimate $= \text{argmax}_{(v_i)} \sum_t \log v_{i_t} - \sum_t \log\left(1 + \sum_{i \in S_t} v_i\right)$

14 (B) - Online Assortment Optimization

So far

- **Module 1:** Allocating resources under sequential demand
  - Uncertainty in future demand
- **Module 2:** Optimizing assortment for a single customer
  - Uncertainty in customer choice

Today

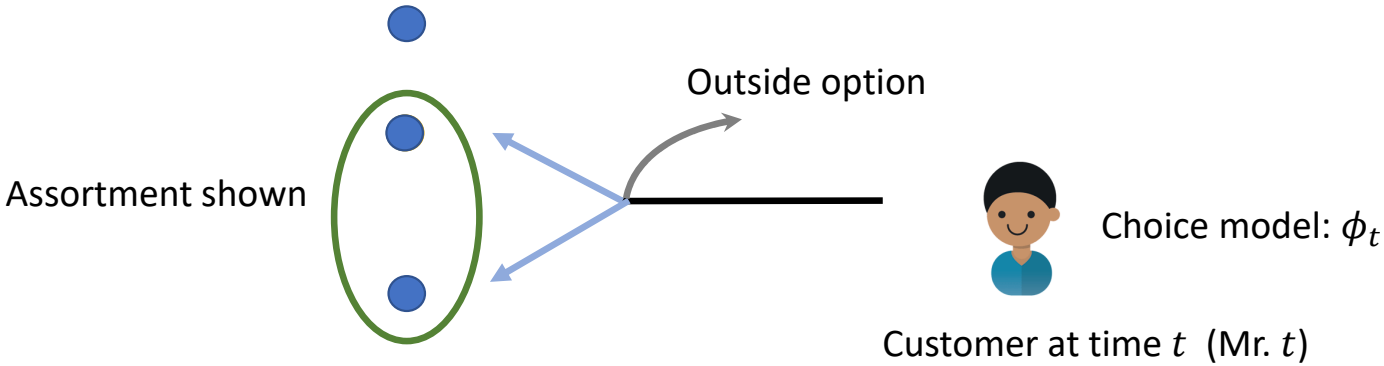
- Module 1: Allocating resources under sequential demand
  - Uncertainty in future demand
- Module 2: Optimizing assortment for a single customer
  - Uncertainty in customer choice

- Optimal assortments under sequential demand
  - Generalize both modules

On many online platforms …

- On arrival of customer, offer a personalized assortment

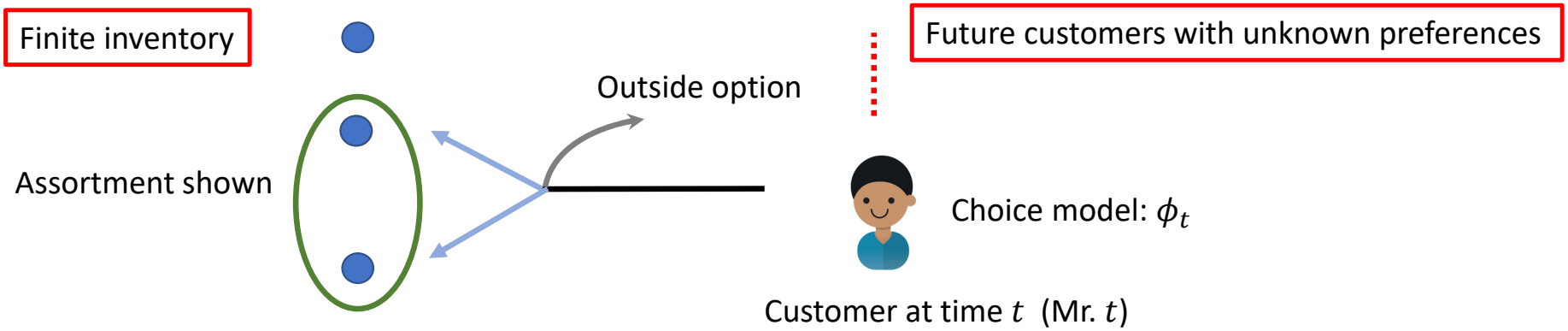
- How? Using choice model
  - Model may be personalized using info from cookies, demographics, history …



On many online platforms …

- On arrival of customer, offer a personalized assortment

- How? Using choice model
  - Model may be personalized using info from cookies, demographics, history …

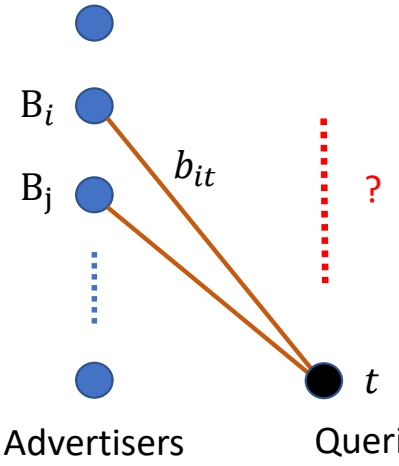


Examples of Personalization

- Each arriving customer t gets their own choice model ϕ_t
  - Based on history: a shopper who buys lots of running shoes is shown latest gear
  - Based on time: a phone user near campus at 8 am is shown coffee and breakfast

- Could also be used primarily for the benefit of the platform
  - Hotel site with 1 “cheap” room left: show it to this visitor, or save it?
    - May decide not to show to visitor who uses Mac (why?!)

Recall: Online Adwords

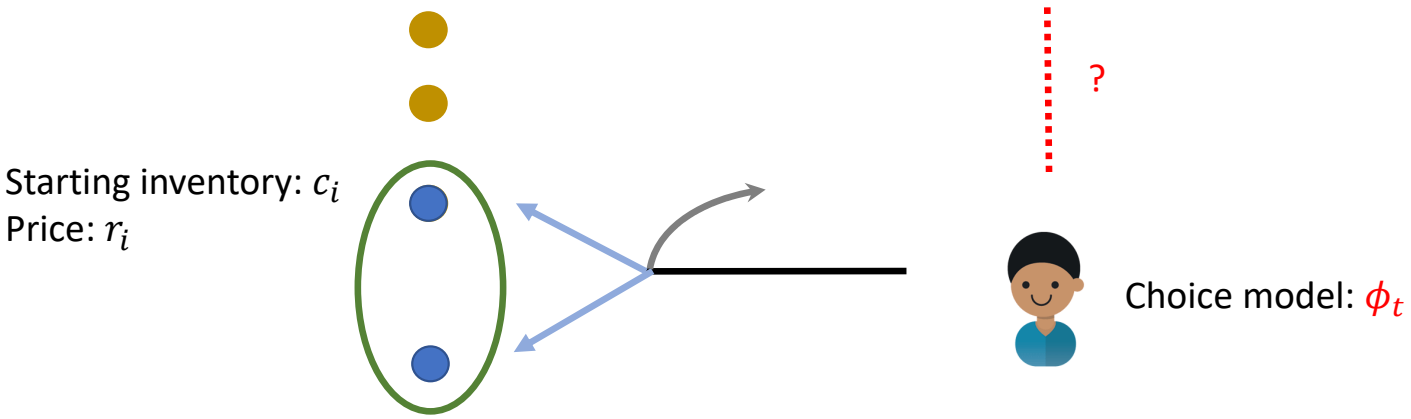


Decision: Match or reject

- Irrevocable
- Future completely unknown

Want to maximize total budget used

Online Assortment Optimization



Decision: Assortment offered

- Irrevocable
- Customer chooses after assortment shown
- Future completely unknown

Want to maximize expected total revenue

Online Assortment vs. Adwords

- **Adwords**: a query arrives → we pick one advertiser (or reject)
  - The match uses the advertiser's budget for sure
- **Online assortment**: a customer arrives → we pick a **set** of products
  - The customer then chooses one product or nothing
  - Inventory is used only if they buy, so revenue is an expected value
- Both: decisions are irrevocable and the future is unknown

Online Assortment Optimization

- Finite inventory of products
- Customers arrive one by one
- We show assortment based on customer type/choice model
  - Personalized choice model may incorporate information about customer
- If customer chooses something, inventory is used

Unknown Future Choice Models

Customer type = Info such as time, location, Mac vs. PC, history etc.

- We may learn a different choice model for each type

Two camps

- **Know nothing** about type of customer until they arrive
- **Stochastic knowledge** of future customer types
  - A distribution over types

Greedy Algorithm

[When we Know Nothing About the Future]

Greedy: To each customer, show?

Greedy Algorithm

[When we Know Nothing About the Future]

Greedy: To each customer, show revenue optimal assortment of available products

- Let $I(t)$ denote the set of available items at t

Show assortment: $S_t = \text{argmax}_{S \subseteq I(t), S \in \mathcal{F}} \sum_{i \in S} r_i \, \phi_t(i, S)$

- $\mathcal{F}$ is the set of feasible assortments

- When ϕ_t is MNL and $\mathcal{F}$ is cardinality constraint, finding S_t is easy (use LP)!

Competitive Ratio

- Greedy is 0.5 competitive for online assortment optimization
- In general, the 0.5 competitive ratio of Greedy is unbeaten!
  - Open problem
- However, for large starting inventory (like the small bid to budget setting), there are better algorithms (similar to Adwords)

A paper I like (not for evaluation):

Golrezaei, Negin and Nazerzadeh, Hamid and Rusmevichientong, Paat, Real-Time Optimization of Personalized Assortments (February 1, 2014). Management Science, 60(6):1532-1551, 2014, Available at SSRN: https://ssrn.com/abstract=2149344 or http://dx.doi.org/10.2139/ssrn.2149344

- Extra time for questions about course project topics